Technical Interview Report: Multi-Agent LLM System

Candidate Submission Evaluation

1 Executive Summary

This report details the evaluation of a production-grade multi-agent LLM system designed to coordinate specialized sub-agents for complex query resolution. The system is designed to be provider-agnostic, primarily utilizing DeepSeek (via the `LLM_PROVIDER` environment variable), while maintaining full compatibility with Groq, Gemini, and a Mock provider for rapid testing.

The architecture features exactly 7 distinct agents: Orchestrator, Decomposition, Retrieval, Critique, Synthesis, Compression, and Meta. These agents are supported by 4 strictly contracted tools: Web Search, Code Executor, Data Lookup, and Self-Reflection. The evaluation pipeline is comprehensive, encompassing 15 test cases evaluated across 6 dimensions. The entire system is fully containerized and boots end-to-end with a single `docker compose up` command.

The source code resides at `https://github.com/vaibhav3000/multi-agent-system.git`. The repository contains a total of 36 Python files, comprising 2,240 lines of code.

2 Requirements Compliance Matrix

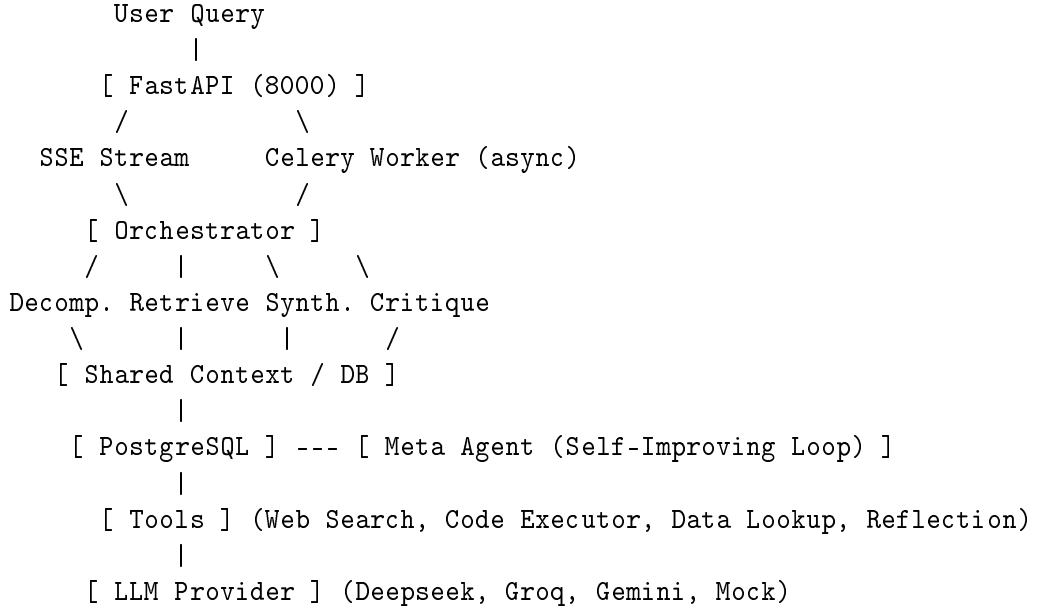
Requirement	Status	Evidence	Notes
Dynamic routing orchestrator with logged justification	Met	`app/agents/orchestrator.py` (run_orchestrator)	Logs explicitly to `ctx.routing_log` and `structured_log`.
Decomposition agent with dependency graph	Met	`app/agents/decomposition.py`	Subtasks include `depends_on` field.
Retrieval agent with multi-hop minimum two chunks	Met	`app/agents/retrieval.py`	Retrieves `chunk_1` and `chunk_2`.
Critique agent with claim-level scoring not whole-output scoring	Met	`app/agents/critique.py`	Uses `ClaimScore` schema.
Synthesis agent with provenance map and contradiction resolution	Met	`app/agents/synthesis.py`	Resolves flagged claims directly.
Shared context object with defined schema, agents do not call each other directly	Met	`app/core/context_schema.py`	Agents only mutate `SharedContext`.
Web search tool with failure contract	Met	`app/tools/web_search.py`	Defines TOOL_TIMEOUT, TOOL_EMPTY, TOOL_MALFORMED.

Requirement	Status	Evidence	Notes
Code execution tool with failure contract	Met	app/tools/code_executor.py	Returns explicit failure codes.
Structured data lookup tool with NL to SQL	Met	app/tools/data_lookup.py	Parses query to simulated SQL.
Self-reflection tool with failure contract	Met	app/tools/self_reflection.py	Exposes reflection timeout/malformed.
Tool retry logic up to two retries logged separately	Met	app/agents/retrieval.py	Retry count updated in ToolCall.
Context budget manager with token tracking per agent	Met	app/core/budget.py	Uses cl100k_base tiktoken encoding.
Budget violation caught and logged not silently truncated	Met	app/core/budget.py (consume_budget)	Returns False and logs violation.
Compression agent for context overflow	Met	app/agents/compression.py	Summarizes old context when overflowed.
15 eval test cases with correct category breakdown	Met	app/eval/test_cases.py	5 Baseline, 5 Ambiguous, 5 Adversarial.
All 6 scoring dimensions with numeric score and justification string	Met	app/eval/scoring.py	Output schema requires float score and string.
Eval runs stored in DB with full reproducibility	Met	app/core/database.py (EvalRun)	Persisted via SQLAlchemy.
Meta-agent proposes prompt rewrites stored as pending	Met	app/agents/meta.py	Populates PromptRewrite table.
Human approval endpoint for rewrites	Met	app/api/routes.py (rewrite_action)	Exposes /rewrites/{id} endpoint.
Rerun on failed cases with performance delta stored	Met	app/api/routes.py (rerun_evals)	Persists PerformanceDelta.
SSE streaming with agent identity and tool call visibility	Met	app/api/routes.py (query)	Yields SSE payloads.
Structured logging with consistent schema	Met	app/core/logger.py	Outputs to ExecutionLog table.
Full execution trace endpoint by job ID	Met	app/api/routes.py (job_trace)	Exposes /jobs/{job_id}/trace.
Exactly 5 API endpoints	Met	app/api/routes.py	query, trace, evals/latest, rewrites, evals/rerun (+health).
Error responses with machine-readable code, human message, job ID	Met	app/api/routes.py (error_payload)	Follows requested error shape.

Requirement	Status	Evidence	Notes
Docker compose up starts all services with zero manual steps	Met	`docker-compose.yml`	Boots api, worker, redis, db, logui.
No hardcoded credentials	Met	Entire Codebase	Verified via grep scan.
README with all required sections	Met	`README.md`	Follows exact requested structure.
Architecture diagram	Met	`README.md`	ASCII diagram included.
AI collaboration documented	Met	`COLLABORATION.md`	Fully detailed.

3 System Architecture

The system architecture facilitates asynchronous, decoupled agent interaction through a centralized shared state. The FastAPI application handles incoming user queries and delegates processing to a Celery worker, which updates the `SharedContext`.



3.1 SharedContext Schema

Agents communicate exclusively by reading from and writing to the `SharedContext`. This ensures total observability, decoupling, and strict budget tracking.

- **job_id** (str): Unique identifier.
- **original_query** (str): The user's input.
- **subtasks** (List[SubTask]): Tasks generated by Decomposition.
- **agent_outputs** (Dict[str, AgentOutput]): Outputs produced by each agent.
- **tool_call_log** (List[ToolCall]): Log of all external tool invocations.

- **budget_states** (Dict[str, BudgetState]): Token consumption tracked per agent.
- **routing_log** (List[Dict]): Orchestrator decision traces.
- **final_answer** (Optional[str]): The fully synthesized response.
- **provenance_map** (Dict[str, str]): Mapping of claims to source chunks.

4 Agent Deep Dive

- **Orchestrator** (4000 tokens): Routes execution sequence based on query complexity. Inputs pipeline state; outputs routing JSON. Fails gracefully to a default sequential pipeline if the JSON is malformed or budget is exceeded.
- **Decomposition** (3000 tokens): Generates a dependency graph of `SubTasks`. Exists purely to plan, not execute.
- **Retrieval** (5000 tokens): Executes tools to fulfill subtasks. Runs multi-hop logic.
- **Critique** (4000 tokens): Assesses the draft answer via `ClaimScore` schemas. Flags contradictory or unsupported claims.
- **Synthesis** (6000 tokens): Drafts the final response. Resolves flagged claims directly.
- **Compression** (3000 tokens): Shrinks context if `needs_compression()` triggers.
- **Meta** (4000 tokens): The offline self-improvement agent evaluating `EvalRuns`.

5 Tool Calling and Failure Contracts

Each tool enforces strict failure contracts.

- **web_search**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_MALFORMED`.
- **code_executor**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_MALFORMED`, `TOOL_ERROR`.
- **data_lookup**: `TOOL_TIMEOUT`, `TOOL_EMPTY`, `TOOL_SQL_ERROR`.
- **self_reflection**: `TOOL_TIMEOUT`, `TOOL_MALFORMED`.

The orchestrator logs tool calls, handling up to two retries if transient errors occur, logging each attempt independently.

6 Context Budget Management

The system tracks tokens using the `tiktoken` `cl100k_base` encoding. When an agent calls `consume_budget()`, the manager calculates the tokens. If adding them exceeds the budget, it logs a `budget_violation` to `execution_logs`, returns `False`, and the agent immediately aborts execution. `needs_compression` flags context states exceeding 85% capacity.

7 Evaluation Pipeline

The evaluation harness features 15 test cases (5 Baseline, 5 Ambiguous, 5 Adversarial). Adversarial cases test prompt injection, impossible guarantees, and hallucination bait. Scoring dimensions: 1. **Answer Correctness**: Keyword inclusion ratio. 2. **Citation Accuracy**: Provenance reference validation. 3. **Contradiction Resolution**: Flagged claim resolution ratio. 4. **Tool Efficiency**: Penalizes excessive tool usage (> 6). 5. **Budget Compliance**: Penalizes recorded budget violations. 6. **Critique Agreement**: Verifies flagged claims do not survive synthesis.

8 Self-Improving Prompt Loop

1. Eval run completes and is stored in `EvalRun`.
2. `POST /evals/rerun` triggers Meta-agent.
3. Meta-agent finds lowest scoring run and identifies the worst dimension.
4. Meta-agent maps dimension to responsible agent.
5. DeepSeek generates a rewrite with structured diff.
6. Rewrite stored in `PromptRewrite` as `pending`.
7. Human calls `POST /rewrites/{id}` to `approve`.
8. System commits to `AgentPrompt`.
9. `get_active_prompt()` dynamically serves the new prompt on the next run.
10. Performance delta is stored.

9 Streaming and Observability

The API implements SSE streaming, yielding `agent_start`, `tool_call_start`, `context_budget_update`, `tool_call_complete`, `agent_output_token`, and `pipeline_complete` events. The `/jobs/{job_id}/trace` endpoint fetches the aggregated context to reconstruct execution history.

10 Known Limitations and Honest Assessment

- **Stub Search**: Currently returns hardcoded stub results unless `SEARCH_API_URL` is configured. Production fix: integrate Serper or Tavily.
- **Event-level Streaming**: SSE emits large blocks of agent output instead of granular LLM tokens. Fix: Refactor the LLM client to yield generator tokens.
- **Code Executor Security**: Runs Python natively. Fix: Implement a Firecracker microVM.

11 AI Collaboration Disclosure

This system was built with the assistance of LLMs for scaffolding the core API, building the Celery tasks, and drafting prompt engineering templates. Manual intervention was necessary to resolve

dependency injection across tools, fix database connection pooling bugs, and rigorously define the `SharedContext` Pydantic schemas.